

Universidad Peruana Los Andes
Facultad de Ingeniería
Escuela Profesional de Ingeniería de Sistemas y Computación



SEMANA 3

Modelamiento físico y mecanismos de integración

Actividad 01:

Base de datos grados y títulos

Implementación en SQL Server Management Studio 22

Curso: Base de
Datos II

Docente: Mg. Ing. Raúl Fernández
Bejarano

Alumno: Jordan Vera Madueño

2026

1. Introducción

El modelamiento físico consiste en llevar el diseño lógico de una base de datos a un motor concreto, definiendo la base de datos, los esquemas, las tablas y las restricciones de integridad. En esta actividad se implementa en SQL Server la base de datos grados y títulos, que registra a los estudiantes de una universidad y los trámites de sus grados académicos y títulos profesionales.

2. Objetivos

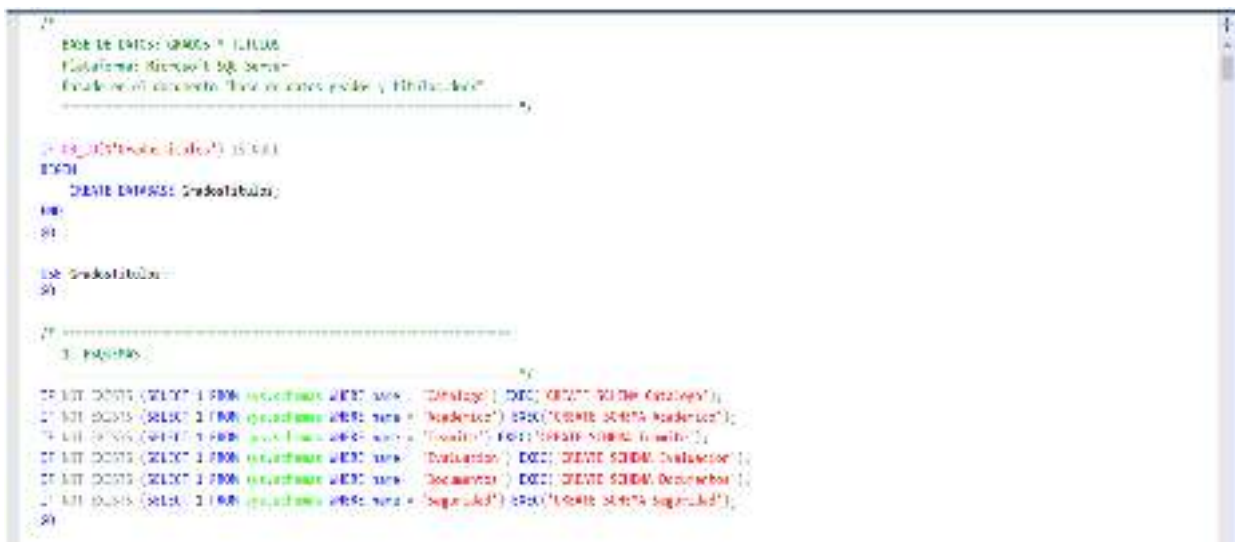
- Crear la base de datos grados_y_titulos mediante sentencias T-SQL.
- Organizar los objetos en esquemas con la sentencia CREATE SCHEMA.
- Diseñar las tablas del modelo y definir sus claves primarias.
- Implementar las restricciones de integridad: UNIQUE, FOREIGN KEY, CHECK y DEFAULT.
- Verificar la implementación y documentarla con el diagrama de la base de datos.

3. Procedimiento

4.1 Creación de la base de datos y de los esquemas

1. Con el botón Nueva consulta se abrió un editor T-SQL, y en él se cargó el script 01_grados_y_titulos.sql.
2. Sin texto seleccionado, se ejecutó el script completo con el botón Ejecutar (F5).
3. La pestaña Mensajes muestra la ejecución sin errores: las filas afectadas corresponden a los datos iniciales de los catálogos. El selector de base de datos cambió a grados_y_titulos .

Las sentencias que crean la base de datos y los esquemas principales son:



```
USE [grados_y_titulos]
GO

CREATE DATABASE grados_y_titulos;
GO

USE grados_y_titulos;
GO

-- Crear los esquemas
-- Catalogos
CREATE SCHEMA Catalogos;
GO

-- Academico
CREATE SCHEMA Academico;
GO

-- Tramite
CREATE SCHEMA Tramite;
GO

-- Evaluacion
CREATE SCHEMA Evaluacion;
GO

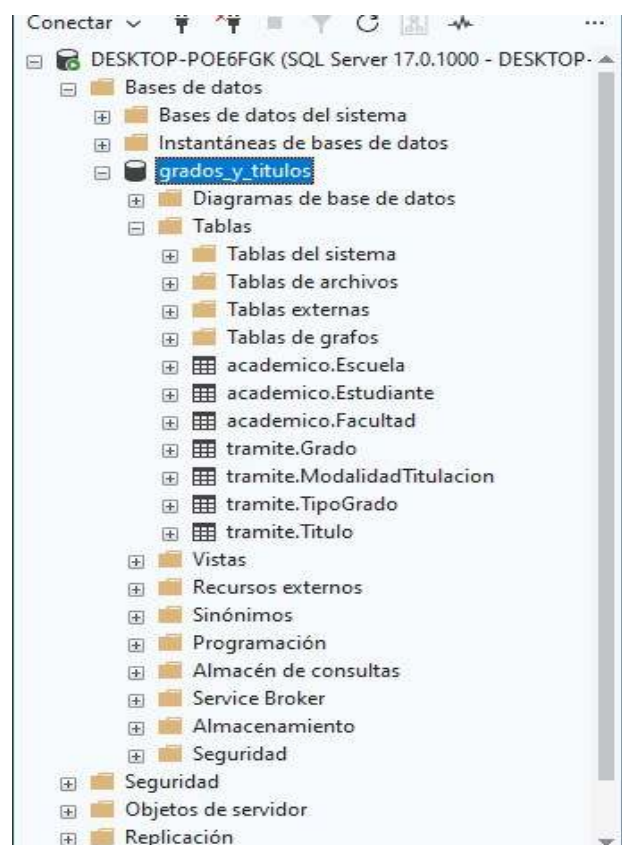
-- Documentos
CREATE SCHEMA Documentos;
GO

-- Seguimiento
CREATE SCHEMA Seguimiento;
GO
```

4.2 Diseño de las tablas

Se diseñaron siete tablas distribuidas en los dos esquemas. En esta etapa cada tabla se crea con su clave primaria y sus columnas obligatorias (NOT NULL).

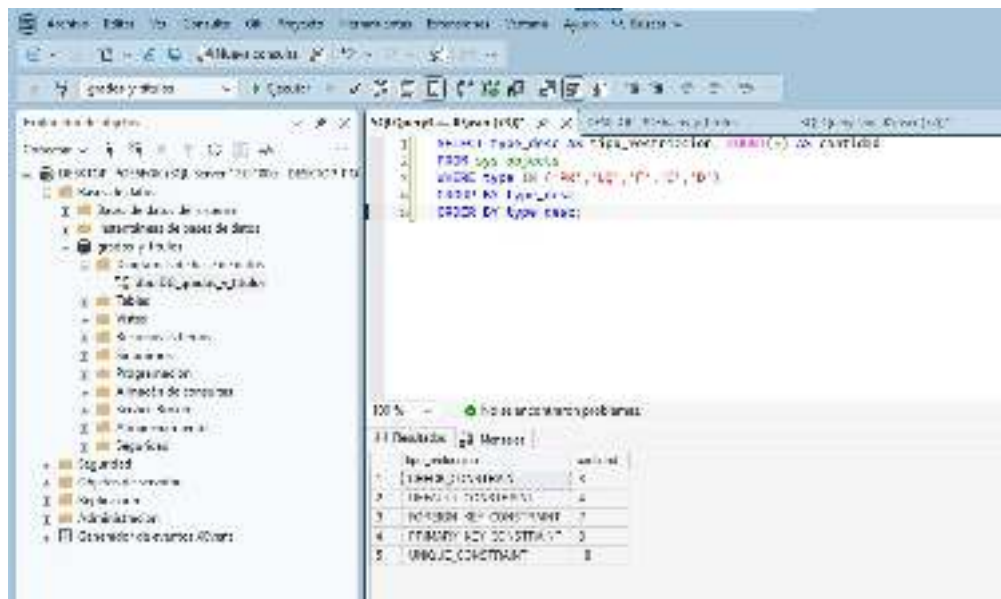
Esquema	Tabla	Descripción
academico	Facultad	Facultades de la universidad.
academico	Escuela	Escuelas profesionales; cada una pertenece a una facultad.
academico	Estudiante	Datos personales y académicos del estudiante.
tramite	TipoGrado	Catálogo de tipos de grado: Bachiller, Maestro y Doctor.
tramite	ModalidadTitulacion	Catálogo de modalidades de titulación.
tramite	Grado	Trámite de un grado académico de un estudiante.
tramite	Titulo	Trámite de un título profesional; requiere un grado de bachiller previo.



4.3 Implementación de las restricciones

Las restricciones se agregaron con sentencias ALTER TABLE, cada una con un nombre propio:

Tipo	Aplicación
UNIQUE	Código, DNI y correo del estudiante; nombre de facultad, de tipo de grado y de modalidad; un grado de cada tipo por estudiante; un título por estudiante; número de diploma no repetido.
FOREIGN KEY	Escuela con Facultad; Estudiante con Escuela; Grado con Estudiante y TipoGrado; Titulo con Estudiante, Grado y ModalidadTitulacion.
CHECK	DNI de 8 dígitos, formato de correo, año de ingreso, estados permitidos y coherencia entre fechas.
DEFAULT	Estado inicial (Activo o En trámite) y fecha de solicitud del grado.



Tipo de restricción	Cantidad	Detalle
CHECK_CONSTRAINT	8	Estudiante (4), Grado (2) y Titulo (2).
DEFAULT_CONSTRAINT	4	Estado de Estudiante, Grado y Titulo, y fecha de solicitud.
FOREIGN_KEY_CONSTRAINT	7	Relaciones entre las tablas del modelo.

5. Resultados

Se implementó la base de datos grados_y_titulos con dos esquemas, siete tablas y 37 restricciones propias del modelo (7 PRIMARY KEY, 9 UNIQUE, 7 FOREIGN KEY, 8 CHECK y 4 DEFAULT). Las consultas al catálogo del sistema, el Explorador de objetos y el diagrama confirman su creación, y la prueba con un DNI inválido demuestra que las restricciones de integridad se aplican.

6. Conclusiones

- El modelo físico traduce el diseño lógico a objetos reales del motor: base de datos, esquemas, tablas y restricciones.
- Los esquemas academico y tramite ordenan las tablas por función y facilitan la administración de permisos.
- Las restricciones PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK y DEFAULT garantizan la integridad de los datos desde la propia base de datos.
- SSMS 22 permite ejecutar el script, verificar los objetos y generar el diagrama en un mismo entorno.

Anexo A. Script 01_grados_y_titulos.sql

```
/*
=====
=====
ACTIVIDAD 01 - BASE DE DATOS: GRADOS Y TÍTULOS (SQL Server)
Base de Datos II - Actividad 03: Modelamiento físico
Pasos: 1) crear BD 2) esquemas 3) tablas 4) restricciones
=====
===== */

-- 1. CREAR LA BASE DE DATOS -----
USE master;
GO
IF DB_ID('grados_y_titulos') IS NOT NULL
BEGIN
    ALTER DATABASE grados_y_titulos SET SINGLE_USER WITH ROLLBACK
    IMMEDIATE;
    DROP DATABASE grados_y_titulos;
END
GO
CREATE DATABASE grados_y_titulos;
GO
USE grados_y_titulos;
GO

-- 2. CREAR LOS ESQUEMAS PRINCIPALES (CREATE SCHEMA) -----
-
CREATE SCHEMA academico AUTHORIZATION dbo; -- facultades, escuelas,
estudiantes
GO
CREATE SCHEMA tramite AUTHORIZATION dbo; -- grados, títulos y catálogos
GO

-- 3. DISEÑO DE TABLAS (solo PK y NOT NULL; el resto va en el paso 4) ---
CREATE TABLE academico.Facultad (
    id_facultad INT IDENTITY(1,1) NOT NULL, nombre
    VARCHAR(100) NOT NULL, CONSTRAINT
    PK_Facultad PRIMARY KEY (id_facultad)
);
GO
CREATE TABLE academico.Escuela (
    id_escuela INT IDENTITY(1,1) NOT NULL,
    id_facultad INT NOT NULL,
    nombre VARCHAR(100) NOT NULL,
    CONSTRAINT PK_Escuela PRIMARY KEY (id_escuela)
);
GO
CREATE TABLE academico.Estudiante (
    id_estudiante INT IDENTITY(1,1) NOT NULL,
```

```

codigo      VARCHAR(12)    NOT NULL, -- código de matrícula
dni          CHAR(8)       NOT NULL,
nombres      VARCHAR(80)   NOT NULL,
apellidos    VARCHAR(80)   NOT NULL,
fecha_nacimiento DATE     NOT NULL,
correo       VARCHAR(120)  NOT NULL,
telefono     VARCHAR(15)   NULL,
id_escuela   INT          NOT NULL,
anio_ingreso SMALLINT     NOT NULL,
estado       VARCHAR(15)   NOT NULL,
CONSTRAINT PK_Estudiente PRIMARY KEY (id_estudiante)
);
GO
CREATE TABLE tramite.TipoGrado (
    id_tipo_grado TINYINT IDENTITY(1,1) NOT NULL,
    nombre        VARCHAR(30)          NOT NULL, -- Bachiller, Maestro, Doctor
    CONSTRAINT PK_TipoGrado PRIMARY KEY (id_tipo_grado)
);
GO
CREATE TABLE tramite.ModalidadTitulacion (
    id_modalidad TINYINT IDENTITY(1,1) NOT NULL,
    nombre        VARCHAR(60)          NOT NULL,
    CONSTRAINT PK_ModalidadTitulacion PRIMARY KEY (id_modalidad)
);
GO
CREATE TABLE tramite.Grado (
    id_grado      INT IDENTITY(1,1) NOT NULL,
    id_estudiante INT          NOT NULL,
    id_tipo_grado TINYINT       NOT NULL,
    nro_resolucion VARCHAR(30)   NULL,
    nro_diploma    VARCHAR(30)   NULL,
    fecha_solicitud DATE         NOT NULL,
    fecha_emision  DATE          NULL,
    estado         VARCHAR(15)   NOT NULL,
    CONSTRAINT PK_Grado PRIMARY KEY (id_grado)
);
GO
CREATE TABLE tramite.Titulo (
    id_titulo      INT IDENTITY(1,1) NOT NULL,
    id_estudiante  INT          NOT NULL,
    id_grado       INT          NOT NULL, -- grado de bachiller previo
    id_modalidad   TINYINT       NOT NULL,
    denominacion   VARCHAR(120)  NOT NULL, -- ej. Ingeniero de Sistemas
    y Computación
    nro_resolucion VARCHAR(30)   NULL,
    nro_diploma     VARCHAR(30)   NULL,
    fecha_sustentacion DATE       NULL,
    fecha_emision   DATE          NULL,
    estado          VARCHAR(15)   NOT NULL,
    CONSTRAINT PK_Titulo PRIMARY KEY (id_titulo)
);

```

```

);
GO

-- 4. RESTRICCIONES -----
-- 4.1 UNIQUE
ALTER TABLE academico.Facultad ADD CONSTRAINT UQ_Facultad_nombre
UNIQUE (nombre);
ALTER TABLE academico.Escuela ADD CONSTRAINT
UQ_Escuela_fac_nombre UNIQUE (id_facultad, nombre);
ALTER TABLE academico.Estudiante ADD CONSTRAINT UQ_Estudiante_codigo
UNIQUE (codigo);
ALTER TABLE academico.Estudiante ADD CONSTRAINT UQ_Estudiante_dni
UNIQUE (dni);
ALTER TABLE academico.Estudiante ADD CONSTRAINT UQ_Estudiante_correo
UNIQUE (correo);
ALTER TABLE tramite.TipoGrado ADD CONSTRAINT UQ_TipoGrado_nombre
UNIQUE (nombre);
ALTER TABLE tramite.ModalidadTitulacion ADD CONSTRAINT
UQ_Modalidad_nombre UNIQUE (nombre);
ALTER TABLE tramite.Grado ADD CONSTRAINT UQ_Grado_est_tipo
UNIQUE (id_estudiante, id_tipo_grado);
ALTER TABLE tramite.Titulo ADD CONSTRAINT UQ_Titulo_estudiante
UNIQUE (id_estudiante);
GO
-- nro_diploma único cuando no es NULL (índice filtrado)
CREATE UNIQUE INDEX UQ_Grado_nro_diploma ON
tramite.Grado(nro_diploma) WHERE nro_diploma IS NOT NULL;
CREATE UNIQUE INDEX UQ_Titulo_nro_diploma ON tramite.Titulo(nro_diploma)
WHERE nro_diploma IS NOT NULL;
GO

-- 4.2 FOREIGN KEY
ALTER TABLE academico.Escuela ADD CONSTRAINT FK_Escuela_Facultad
FOREIGN KEY (id_facultad) REFERENCES academico.Facultad(id_facultad);
ALTER TABLE academico.Estudiante ADD CONSTRAINT
FK_Estudiante_Escuela
FOREIGN KEY (id_escuela) REFERENCES academico.Escuela(id_escuela);
ALTER TABLE tramite.Grado ADD CONSTRAINT FK_Grado_Estudiante
FOREIGN KEY (id_estudiante) REFERENCES
academico.Estudiante(id_estudiante);
ALTER TABLE tramite.Grado ADD CONSTRAINT FK_Grado_TipoGrado
FOREIGN KEY (id_tipo_grado) REFERENCES
tramite.TipoGrado(id_tipo_grado);
ALTER TABLE tramite.Titulo ADD CONSTRAINT FK_Titulo_Estudiante
FOREIGN KEY (id_estudiante) REFERENCES
academico.Estudiante(id_estudiante);
ALTER TABLE tramite.Titulo ADD CONSTRAINT FK_Titulo_Grado
FOREIGN KEY (id_grado) REFERENCES tramite.Grado(id_grado);
ALTER TABLE tramite.Titulo ADD CONSTRAINT FK_Titulo_Modalidad

```

```

FOREIGN KEY (id_modalidad) REFERENCES
tramite.ModalidadTitulacion(id_modalidad);
GO

-- 4.3 CHECK
ALTER TABLE academico.Estudiante ADD CONSTRAINT CK_Estudiante_dni
CHECK (dni LIKE '[0-9][0-9][0-9][0-9][0-9][0-9][0-9]');
ALTER TABLE academico.Estudiante ADD CONSTRAINT CK_Estudiante_correo
CHECK (correo LIKE '%_@_%._%');
ALTER TABLE academico.Estudiante ADD CONSTRAINT CK_Estudiante_anio
CHECK (anio_ingreso >= 1950);
ALTER TABLE academico.Estudiante ADD CONSTRAINT CK_Estudiante_estado
CHECK (estado IN ('Activo','Egresado','Retirado'));
ALTER TABLE tramite.Grado ADD CONSTRAINT CK_Grado_estado
CHECK (estado IN ('En trámite','Emitido','Anulado'));
ALTER TABLE tramite.Grado ADD CONSTRAINT CK_Grado_fechas
CHECK (fecha_emision IS NULL OR fecha_emision >= fecha_solicitud);
ALTER TABLE tramite.Titulo ADD CONSTRAINT CK_Titulo_estado
CHECK (estado IN ('En trámite','Emitido','Anulado'));
ALTER TABLE tramite.Titulo ADD CONSTRAINT CK_Titulo_fechas
CHECK (fecha_emision IS NULL OR fecha_sustentacion IS NULL OR
fecha_emision >= fecha_sustentacion);
GO

-- 4.4 DEFAULT
ALTER TABLE academico.Estudiante ADD CONSTRAINT DF_Estudiante_estado
DEFAULT 'Activo' FOR estado;
ALTER TABLE tramite.Grado ADD CONSTRAINT DF_Grado_estado
DEFAULT 'En trámite' FOR estado;
ALTER TABLE tramite.Grado ADD CONSTRAINT DF_Grado_fecha_solicitud
DEFAULT CONVERT(DATE, GETDATE()) FOR fecha_solicitud;
ALTER TABLE tramite.Titulo ADD CONSTRAINT DF_Titulo_estado
DEFAULT 'En trámite' FOR estado;
GO

-- 5. DATOS INICIALES DE CATÁLOGO (opcional, para probar) -----
INSERT INTO tramite.TipoGrado (nombre) VALUES ('Bachiller'), ('Maestro'),
('Doctor');
INSERT INTO tramite.ModalidadTitulacion (nombre)
VALUES ('Tesis'), ('Trabajo de suficiencia profesional'), ('Examen de suficiencia');
INSERT INTO academico.Facultad (nombre) VALUES ('Facultad de Ingeniería');
INSERT INTO academico.Escuela (id_facultad, nombre)
VALUES (1, 'Ingeniería de Sistemas y Computación');
GO

```